

## UNIT 3: Testing Techniques

### Topic 1.1: Boundary Value Analysis (BVA)

#### Introduction:

Boundary Value Analysis (BVA) is a software testing technique that focuses on testing the "edges" or boundaries of input domains. It's based on the idea that errors are more likely to occur at the boundaries of valid and invalid input ranges. BVA is often used in conjunction with Equivalence Partitioning (EP).

#### Key Concepts:

- **Boundaries:** These are the minimum, maximum, and values just above/below these extremes for a given input.
- **Valid Boundary Values:** Values that are within the acceptable range.
- **Invalid Boundary Values:** Values that are outside the acceptable range.

#### Principle:

If an input field accepts values between X and Y, then BVA would involve testing:

- X (minimum valid)
- X-1 (just below minimum, invalid)
- Y (maximum valid)
- Y+1 (just above maximum, invalid)
- Sometimes, X+1 and Y-1 (values just inside the valid range) are also included for more thorough testing.

#### Advantages:

- Effective in finding defects at the boundaries.
- Systematic and easy to apply.
- Reduces the number of test cases compared to exhaustive testing.

#### Disadvantages:

- May not be effective for all types of inputs (e.g., non-numeric).
- Can be time-consuming for systems with many input fields.

#### Example:

- **Input Field:** Age
- **Valid Range:** 18 to 65
- **BVA Test Cases:**
  - 17 (Invalid)
  - 18 (Valid)
  - 19 (Valid, sometimes included)
  - 64 (Valid, sometimes included)
  - 65 (Valid)

- 66 (Invalid)

## **Topic 1.2: Equivalence Partitioning (EP)**

### **Introduction:**

Equivalence Partitioning (EP) is a black-box testing technique that divides the input domain of a software component into "equivalence classes" or partitions. The idea is that if a test case from a partition detects a defect, other test cases from the same partition are likely to detect the same defect.

### **Key Concepts:**

- **Equivalence Class/Partition:** A set of inputs that are expected to behave in the same way.
- **Valid Equivalence Class:** A partition containing valid inputs.
- **Invalid Equivalence Class:** A partition containing invalid inputs.

### **Principle:**

Select one representative test case from each equivalence class. This single test case is assumed to be representative of the entire class.

### **Advantages:**

- Significantly reduces the number of test cases.
- Helps in covering a wide range of inputs with fewer tests.
- Effective for detecting common defects related to input processing.

### **Disadvantages:**

- Relies on accurate identification of equivalence classes, which can be challenging for complex systems.
- May miss defects that occur at the boundaries (addressed by BVA).

### **Example:**

- **Input Field:** Number of items in an online order
- **Requirements:**
  - Minimum 1 item
  - Maximum 100 items
  - Cannot be zero or negative
  - Cannot be more than 100
- **Equivalence Classes:**
  - **Valid:**
    - 1-100 (e.g., select 50)
  - **Invalid:**
    - Less than 1 (e.g., 0, -5)
    - Greater than 100 (e.g., 101, 150)
    - Non-numeric (e.g., "abc", "!@#")

- **EP Test Cases (one from each class):**

- 50
- 0
- 101
- "abc"

### **Topic 1.3: Decision Table Testing**

#### **Introduction:**

Decision Table testing is a black-box testing technique that is suitable for functionalities where the system's behavior depends on a combination of multiple conditions. It represents complex business rules in a tabular form, making it easier to identify all possible combinations of conditions and their corresponding actions.

Structure of a Decision Table:

A decision table typically has four parts:

1. **Conditions Stubs:** List all conditions relevant to the decision.
2. **Action Stubs:** List all actions that can be taken.
3. **Condition Entries:** Specify the state (True/False, Yes/No, specific value) of each condition for each rule.
4. **Action Entries:** Indicate which actions are performed for each rule.

#### **Steps to Create a Decision Table:**

1. Identify all conditions that influence the behavior.
2. Identify all possible actions.
3. Determine the number of rules (combinations of conditions). If there are 'n' conditions, there can be up to  $2^n$  rules (though not all may be valid or unique).
4. Fill in the condition entries for each rule.
5. Determine the corresponding actions for each rule.
6. Simplify the table by combining rules if possible (e.g., using "don't care" symbols).

#### **Advantages:**

- Excellent for testing complex business logic.
- Ensures all combinations of conditions are considered.
- Helps in identifying ambiguities or missing rules in requirements.
- Clear and easy to understand.

#### **Disadvantages:**

- Can become very large and complex for a high number of conditions.

- Time-consuming to create for systems with many interconnected conditions.

#### Example:

- **Scenario:** Online order discount.
- **Conditions:**
  - Is customer a premium member? (Yes/No)
  - Is order total > \$100? (Yes/No)
- **Actions:**
  - Apply 10% discount
  - Apply 5% discount
  - No discount

| Rule | Condition:<br>Premium<br>Member? | Condition:<br>Order Total<br>> \$100? | Action:<br>Apply 10%<br>Discount | Action:<br>Apply 5%<br>Discount | Action: No<br>Discount |
|------|----------------------------------|---------------------------------------|----------------------------------|---------------------------------|------------------------|
| 1    | No                               | No                                    |                                  |                                 | X                      |
| 2    | No                               | Yes                                   |                                  | X                               |                        |
| 3    | Yes                              | No                                    | X                                |                                 |                        |
| 4    | Yes                              | Yes                                   | X                                |                                 |                        |

### Topic 1.4: State Transition Testing

#### Introduction:

State Transition testing is a black-box testing technique that models the behavior of a system in terms of states, transitions between those states, and the events that trigger those transitions. It's particularly useful for systems with a finite number of states and complex state-dependent behavior.

#### Key Concepts:

- **State:** A particular condition or mode that a system can be in at a given time.
- **Event:** Something that happens that causes a transition from one state to another (e.g., user action, system input).
- **Transition:** A change from one state to another, triggered by an event.
- **Action:** An activity performed by the system when a transition occurs.

**State Transition Diagram:**

A graphical representation of the states, events, and transitions.

- States are represented by nodes (circles/rectangles).
- Transitions are represented by arrows connecting states.
- Events triggering transitions are labeled on the arrows.

**Steps for State Transition Testing:**

1. Identify all possible states of the system or component.
2. Identify all events that can cause a change of state.
3. Draw a state transition diagram to visualize the states and transitions.
4. Create test cases to cover:
  - All valid transitions.
  - Invalid transitions (events that should *not* cause a state change).
  - All states being visited at least once.
  - Specific sequences of states and events.

**Advantages:**

- Good for testing systems with complex workflows and state-dependent behavior.
- Helps in identifying "dead" states or unreachable states.
- Can expose defects related to incorrect state changes or missing transitions.
- Provides clear visual representation of system behavior.

**Disadvantages:**

- Can be complex for systems with a large number of states and transitions.
- Requires a clear understanding of system behavior and state definitions.

**Example:**

- **Scenario:** A traffic light system.
- **States:** Red, Green, Yellow
- **Events:** Timer expires (various durations)

(Diagram would show Red -> Green -> Yellow -> Red transitions triggered by timer events)

**Topic 1.5: Use Case Testing****Introduction:**

Use Case testing is a black-box testing technique derived directly from Use Cases. Use Cases describe how a user interacts with a system to achieve a specific goal. This testing approach focuses on validating the functionality from an end-user perspective, ensuring that the system behaves as expected for common and alternative user flows.

### Key Concepts:

- **Use Case:** A description of a set of sequences of actions, including variants, that a system performs to yield an observable result of value to an actor.
- **Actor:** A user or another system that interacts with the system under test.
- **Main Flow/Basic Flow:** The primary, successful path through the use case.
- **Alternative Flows:** Variations of the main flow, including optional steps or different outcomes.
- **Exceptional Flows:** Error conditions or situations where the system cannot complete the main goal.

### Steps for Use Case Testing:

1. Identify relevant Use Cases for the system or feature.
2. For each Use Case, identify the main flow, alternative flows, and exceptional flows.
3. Translate each flow into one or more test cases.
4. Define preconditions (what must be true before the use case starts) and postconditions (what should be true after the use case completes).
5. Identify data required for each test case.

### Advantages:

- Focuses on business processes and user goals.
- Provides excellent test coverage from an end-user perspective.
- Helps in identifying integration issues between different modules.
- Easy for business stakeholders to understand and review.
- Can be used early in the development lifecycle (even before code is written).

### Disadvantages:

- Requires well-defined and detailed Use Cases.
- May not cover all technical aspects or internal system logic (complemented by other techniques).
- Can be time-consuming to create detailed test cases for all flows.

### Example:

- **Use Case:** Place an Order (E-commerce)
- **Main Flow:**
  1. User adds items to cart.
  2. User proceeds to checkout.
  3. User enters shipping information.
  4. User enters payment information.
  5. User confirms order.
  6. System processes payment and confirms order.

- **Alternative Flow 1:** User applies a discount code.
- **Alternative Flow 2:** User logs in during checkout.
- **Exceptional Flow 1:** Invalid credit card details.
- **Exceptional Flow 2:** Item out of stock during checkout.

## Module 2: Software Testing Process & Defect Management

### Topic 2.1: Defect Life Cycle (Bug Life Cycle)

#### Introduction:

The Defect Life Cycle (DLC), often called the Bug Life Cycle, describes the entire journey of a defect from its discovery to its closure. Understanding the DLC is crucial for efficient defect management and ensures that defects are tracked, fixed, and retested effectively.

#### Typical Stages of a Defect:

1. **New:** The defect is identified and logged for the first time.
2. **Assigned:** The defect is assigned to a developer or development team to be fixed.
3. **Open/Active:** The developer is working on fixing the defect.
4. **Fixed:** The developer has implemented a fix for the defect.
5. **Pending Retest:** The defect fix has been deployed to the test environment, and it's awaiting retesting by the QA team.
6. **Retest:** The QA team retests the defect to verify the fix.
7. **Closed:** The defect is confirmed as fixed and verified by QA. No further action is required.
8. **Reopened:** If the defect is not fixed correctly or the fix introduces new issues during retest, it is reopened and sent back to the developer.
9. **Rejected/Deferred:**
  - **Rejected:** The defect is deemed not a defect (e.g., "works as designed," duplicate, not reproducible).
  - **Deferred:** The defect is acknowledged but postponed for a future release due to lower priority or time constraints.

#### Key Roles in the DLC:

- **Tester:** Identifies, logs, retests, and verifies defects.
- **Developer:** Analyzes, fixes, and updates the defect status.
- **Lead/Manager:** Reviews, assigns, and prioritizes defects.

#### Importance of DLC:

- Provides a structured approach to managing defects.
- Ensures clear communication and accountability.
- Helps in tracking the progress of defect resolution.
- Improves overall software quality by ensuring defects are addressed systematically.

## Topic 2.2: Bug Tracking System

### Introduction:

A Bug Tracking System (BTS), also known as a Defect Tracking System or Issue Tracking System, is a software application that helps teams log, track, manage, and report on software bugs and issues. It centralizes all defect-related information, facilitating collaboration and streamlining the defect management process.

### Key Features of a Bug Tracking System:

- **Defect Logging:** Ability to create new defect records with details like title, description, steps to reproduce, expected vs. actual results, severity, priority, environment, attachments (screenshots, logs).
- **Status Management:** Tracking the current state of a defect through its life cycle (New, Assigned, Fixed, Closed, etc.).
- **Assignment & Workflow:** Assigning defects to specific individuals or teams and defining workflows for defect resolution.
- **Prioritization & Severity:** Classifying defects based on their impact (severity) and urgency (priority).
- **Reporting & Analytics:** Generating reports on defect trends, open/closed defects, defect density, resolution times, etc.
- **Search & Filtering:** Easily finding specific defects based on various criteria.
- **Notifications:** Automatic alerts for status changes, assignments, comments.
- **Collaboration:** Commenting, discussions, and attachments to facilitate communication.
- **History/Audit Trail:** Maintaining a complete history of all changes made to a defect.
- **Integration:** Integration with other tools like version control, test management, and CI/CD pipelines.

### Benefits of Using a BTS:

- **Improved Communication:** Centralized information reduces miscommunication.
- **Enhanced Collaboration:** Developers, testers, and project managers work together effectively.
- **Better Visibility:** Stakeholders have real-time insight into product quality.
- **Efficient Tracking:** Defects are rarely lost or overlooked.
- **Data for Improvement:** Metrics from the BTS can be used for process improvement.
- **Accountability:** Clear ownership for defect resolution.

### Popular Bug Tracking Systems:

- Jira (Atlassian)
- Bugzilla
- Redmine
- Azure DevOps
- MantisBT

- Trello (can be adapted)

## Topic 2.3: Logging Defects in JIRA

### Introduction:

Jira is one of the most widely used issue tracking and project management tools, extensively used for bug tracking. Logging defects effectively in Jira is crucial for clear communication and efficient resolution.

### Steps to Log a Defect (Bug) in Jira:

1. **Access Jira:** Log in to your Jira instance.
2. **Navigate to Project:** Select the relevant project where you want to log the bug.
3. **Click "Create" / "Create Issue":**
  - Look for a "Create" button or "Create Issue" option, usually prominent on the dashboard or project view.
4. **Select Issue Type:** Choose "Bug" from the "Issue Type" dropdown.
5. **Project:** Ensure the correct project is selected.
6. **Summary (Short Description/Title):**
  - Provide a concise and clear summary that quickly conveys the essence of the bug.
  - *Example:* "Login button not responsive on mobile browsers."
7. **Description (Detailed Information):**
  - **Steps to Reproduce:** A numbered list of precise steps to reproduce the bug consistently.
    - *Example:*
      1. Go to [www.example.com](http://www.example.com) on a Chrome mobile browser.
      2. Enter valid username "testuser" and password "password123".
      3. Click the "Login" button.
  - **Expected Result:** What the system should do if there were no bug.
    - *Example:* "User should be logged in and redirected to the dashboard."
  - **Actual Result:** What actually happened, describing the bug's manifestation.
    - *Example:* "The login button appears clickable but nothing happens on click. User remains on the login page."
  - **Environment:** Specify the operating system, browser, device, build number, URL, etc., where the bug was observed.
    - *Example:* OS: Android 12, Browser: Chrome 103, Device: Samsung S22, Build: v1.2.0, URL: [staging.example.com](http://staging.example.com)
  - **Attachments:** Attach screenshots, video recordings, error logs, or relevant console output. This is *critical* for developers.
8. **Priority:**
  - Indicates the urgency of fixing the bug (e.g., Highest, High, Medium, Low, Lowest).

Often set by the QA Lead/Project Manager.

**9. Severity:**

- Indicates the impact of the bug on the system's functionality (e.g., Blocker, Critical, Major, Minor, Trivial).

**10. Assignee:**

- Assign the bug to the relevant developer or the development team lead. If unsure, leave it unassigned for triage.

**11. Reporter:**

- Your name should automatically populate here.

**12. Labels (Optional):** Add relevant tags for categorization (e.g., "UI", "Frontend", "Payment", "Regression").

**13. Components (Optional):** If the project uses components, select the relevant one (e.g., "User Management", "Shopping Cart").

**14. Link Issues (Optional):** If related to other bugs, tasks, or user stories, link them.

**15. Click "Create":** Submit the bug report.

**Best Practices for Logging Defects:**

- **Reproducible:** Ensure the bug can be consistently reproduced. If not, state it's intermittent and provide details.
- **Clear & Concise:** Use simple, unambiguous language.
- **Specific:** Avoid vague statements.
- **One Bug Per Report:** Log separate bugs for distinct issues.
- **Attach Evidence:** Always include screenshots, videos, or logs.
- **Prioritize & Severity:** Assign appropriate priority and severity.
- **Neutral Language:** Describe facts, not opinions.

**Module 3: Test Case Writing & End-to-End Design**

**Topic 3.1: Writing Test Cases using BVA and EP**

**Integrating BVA and EP in Test Case Design:**

BVA and EP are often used together to create a robust set of test cases. EP helps reduce the overall number of test cases by dividing inputs into logical groups, while BVA then focuses on the critical boundary values within those groups.

**Steps to Write Test Cases using BVA & EP:**

- 1. Identify Input Fields/Parameters:** List all input fields, parameters, or conditions for the feature under test.
- 2. Apply Equivalence Partitioning (EP):**
  - For each input, divide its possible values into valid and invalid equivalence classes.
  - Select one representative value from each equivalence class. These become your initial test cases.

### 3. Apply Boundary Value Analysis (BVA):

- For each identified equivalence class (especially numeric or ordered ones), identify its boundaries.
- Generate test cases for values at, just below, and just above these boundaries (e.g., min, min-1, min+1, max, max-1, max+1).

### 4. Combine and Refine:

- Combine the test cases generated from EP and BVA.
- Remove duplicate test cases.
- Ensure each test case has a unique identifier.
- Define clear test steps, expected results, and preconditions.
- Consider negative test cases (invalid inputs) generated by both techniques.

### Test Case Structure:

| Test Case ID | Feature/Module | Input Field | Test Data (Value) | Technique Used (EP/BVA) | Preconditions | Steps to Execute              | Expected Result                               | Pass/Fail | Notes |
|--------------|----------------|-------------|-------------------|-------------------------|---------------|-------------------------------|-----------------------------------------------|-----------|-------|
| TC-001       | Age Validation | Age         | 18                | BVA                     | N/A           | Enter 18 in age field, Submit | System accepts age 18                         |           |       |
| TC-002       | Age Validation | Age         | 17                | BVA                     | N/A           | Enter 17 in age field, Submit | Error message "Age must be between 18 and 65" |           |       |

|        |                |          |    |               |               |                               |                              |  |  |
|--------|----------------|----------|----|---------------|---------------|-------------------------------|------------------------------|--|--|
| TC-003 | Age Validation | Age      | 40 | EP (Valid)    | N/A           | Enter 40 in age field, Submit | System accepts age 40        |  |  |
| TC-004 | Items in Cart  | Quantity | 50 | EP (Valid)    | Item in stock | Enter 50, Add to Cart         | 50 items added successfully  |  |  |
| TC-005 | Items in Cart  | Quantity | 0  | BVA (Invalid) | N/A           | Enter 0, Add to Cart          | Error: "Min 1 item required" |  |  |

#### Example: E-commerce Product Quantity Field

- **Requirement:** Users can add between 1 and 99 items of a product to the cart.
- **Equivalence Classes (EP):**
  - Valid: 1-99 (e.g., 50)
  - Invalid (Less than 1): 0, -5
  - Invalid (Greater than 99): 100, 150
  - Invalid (Non-numeric): "abc"
- **Boundary Values (BVA):**
  - Min boundary: 1 (valid), 0 (invalid), 2 (valid - sometimes included)
  - Max boundary: 99 (valid), 100 (invalid), 98 (valid - sometimes included)

- **Combined Test Cases:**

| TC ID  | Input Field | Test Data | Technique | Expected Result                                   |
|--------|-------------|-----------|-----------|---------------------------------------------------|
| TC-001 | Quantity    | 1         | BVA       | Product added to cart with quantity 1             |
| TC-002 | Quantity    | 0         | BVA       | Error message: "Quantity must be at least 1."     |
| TC-003 | Quantity    | 99        | BVA       | Product added to cart with quantity 99            |
| TC-004 | Quantity    | 100       | BVA       | Error message: "Quantity cannot exceed 99."       |
| TC-005 | Quantity    | 50        | EP        | Product added to cart with quantity 50            |
| TC-006 | Quantity    | -5        | EP        | Error message: "Quantity must be at least 1."     |
| TC-007 | Quantity    | "abc"     | EP        | Error message: "Quantity must be a valid number." |

### Topic 3.2: End-to-End Test Design for a Sample Feature (e.g., E-commerce Cart)

#### Introduction:

End-to-End (E2E) testing validates the entire software system from start to finish, simulating real-world user scenarios. It ensures that all integrated components of an application work together correctly and meet business requirements. For an E-commerce Cart, this involves testing the entire user journey from adding items to successful checkout.

#### Goals of E2E Testing:

- Validate critical user workflows.
- Ensure data integrity across integrated systems.
- Identify issues in integrations between modules (frontend, backend, database, payment gateways, shipping APIs, etc.).
- Confirm the system meets business objectives.

#### Steps for End-to-End Test Design (E-commerce Cart Example):

##### 1. Understand the Feature/System Flow:

- Review requirements, user stories, and design documents related to the E-commerce Cart.
- Map out the complete user journey: Browse Products -> Add to Cart -> View Cart -> Checkout -> Payment -> Order Confirmation.
- Identify all interacting components/systems (Product Catalog, Inventory, User Authentication, Payment Gateway, Shipping Service, Order Management System, Database).

##### 2. Define Scope and Critical Paths:

- What are the most important user flows for the cart? (e.g., adding a single item, adding multiple items, applying discounts, removing items, changing quantities, guest checkout, logged-in checkout).
- Prioritize these flows for E2E testing.

##### 3. Identify Actors and Data:

- **Actors:** Registered user, Guest user, Admin (for stock updates if applicable).
- **Test Data:**
  - Test user accounts (with various roles/privileges).
  - Products (in stock, out of stock, discounted, different categories).
  - Payment methods (valid credit cards, invalid cards, different types like PayPal).
  - Shipping addresses (valid, invalid).
  - Coupon codes (valid, expired, invalid).

##### 4. Break Down into Scenarios/Test Cases:

- **High-Level Scenarios:** Describe the overall user journey.
- **Detailed Test Cases:** For each scenario, break it down into specific steps,

preconditions, and expected results. Apply various testing techniques (EP, BVA, Decision Tables) where applicable within these steps.

## 5. Design Test Cases for E-commerce Cart (Sample Scenarios):

### ○ Scenario 1: Basic Product Addition & Checkout (Happy Path - Guest User)

■ **Preconditions:** Available product, Guest user not logged in.

■ **Steps:**

1. Browse to Product Page.
2. Select a product and quantity (e.g., 1, using EP/BVA for quantity).
3. Click "Add to Cart".
4. Verify success message/cart icon update.
5. Navigate to Cart Page.
6. Verify correct product, quantity, and subtotal.
7. Click "Proceed to Checkout".
8. Select "Guest Checkout".
9. Enter valid Shipping Information.
10. Enter valid Payment Information (e.g., Credit Card).
11. Click "Place Order".
12. Verify Order Confirmation Page displays correct order details.
13. Verify email confirmation sent (if applicable).

■ **Expected Results:** All steps executed successfully, data consistent throughout the flow.

### ○ Scenario 2: Apply Discount Code & Checkout (Logged-in User)

■ **Preconditions:** Registered user account, Valid discount code, Product in cart.

■ **Steps:**

1. Login as a registered user.
2. Add products to cart.
3. Navigate to Cart Page.
4. Enter a valid discount code (Test Cases for valid/invalid/expired codes from Decision Table/EP).
5. Verify discount applied and total recalculated.
6. Proceed to Checkout.
7. Select existing shipping/billing address or enter new.
8. Select payment method.
9. Place Order.
10. Verify order confirmation with discounted price.

### ○ Scenario 3: Quantity Updates & Item Removal in Cart

■ **Preconditions:** Product(s) in cart.

■ **Steps:**

1. Add multiple quantities of a product (e.g., 5).
2. Go to Cart Page.
3. Change quantity to a valid value (e.g., 2) (BVA/EP: 1, 99).

4. Verify total updates.
  5. Change quantity to an invalid value (e.g., 0, 100) (BVA/EP).
  6. Verify error message.
  7. Remove an item from the cart.
  8. Verify item removed and total updated.
- **Scenario 4: Out of Stock Product Handling**
    - **Preconditions:** A product is configured as "Out of Stock."
    - **Steps:**
      1. Try to add an out-of-stock product to the cart.
      2. Verify appropriate message (e.g., "Out of Stock," "Cannot add").
      3. (Alternative) Add an in-stock product, then update its stock to 0 while in cart.
      4. Proceed to checkout.
      5. Verify system prevents checkout or notifies about out-of-stock item.
  - **Scenario 5: Invalid Payment Information**
    - **Preconditions:** Product in cart.
    - **Steps:**
      1. Proceed to checkout.
      2. Enter invalid credit card number/expiry/CVV.
      3. Attempt to place order.
      4. Verify appropriate error message from payment gateway integration.
6. **Consider Edge Cases and Error Handling:**
    - Empty cart scenarios.
    - Session timeouts.
    - Network disconnections during checkout.
    - Concurrent updates (e.g., two users trying to buy the last item).
    - Browser back/forward button behavior.
  7. **Identify Test Environment and Tools:**
    - **Environment:** Staging/Pre-production environment that closely mimics production.
    - **Tools:**
      - Manual testing (for exploratory and complex scenarios).
      - Automation tools (Selenium, Playwright, Cypress) for repetitive E2E flows.
      - API testing tools (Postman) for backend interactions, if needed.
      - Browser developer tools for network/console errors.
  8. **Execution and Reporting:**
    - Execute the designed test cases.
    - Log any defects found in Jira (as per Topic 2.3).
    - Track progress and report on E2E test coverage and results.

### **Key Considerations for E2E Test Design:**

- **Realistic Scenarios:** Design tests that reflect how users actually interact with the system.

- **Data Management:** Plan for test data setup and teardown.
- **Environment Stability:** Ensure the E2E test environment is stable and representative.
- **Maintainability:** Design modular and understandable tests.
- **Collaboration:** Work closely with developers, product owners, and other stakeholders.